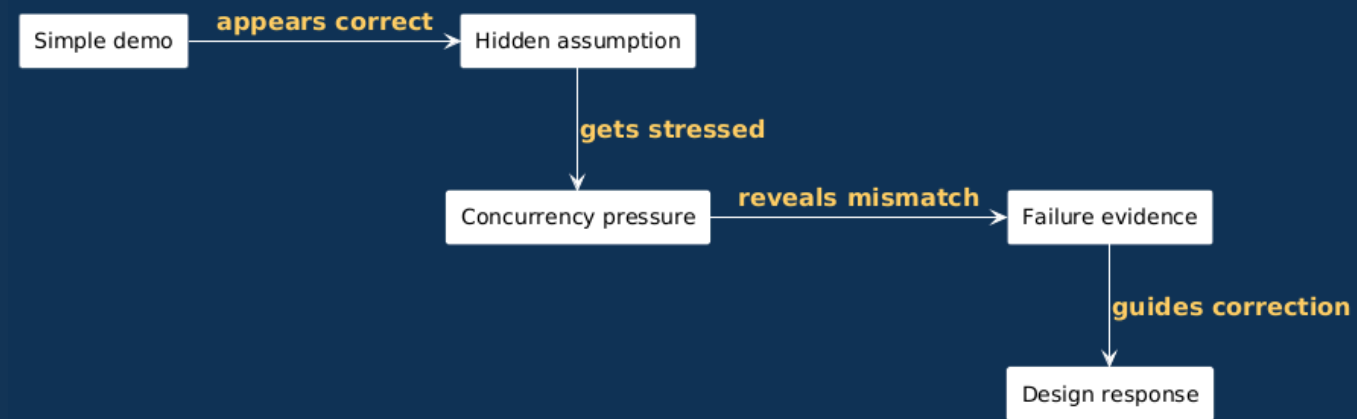


SIMPLICITY VS. SCALABILITY

LECTURE FOCUS

- Simplicity vs. scalability is a design pressure, not just a slogan.
- The key question is whether a system still preserves its invariant under pressure.
- Pressure can come from concurrency, retries, delayed events, or resource limits.
- A design should make the invariant visible and the failure reproducible.
- Evidence matters: logs, traces, interleavings, and repeatable tests.
- The final goal is a defensible design choice for A2 and the Integrity Packet.

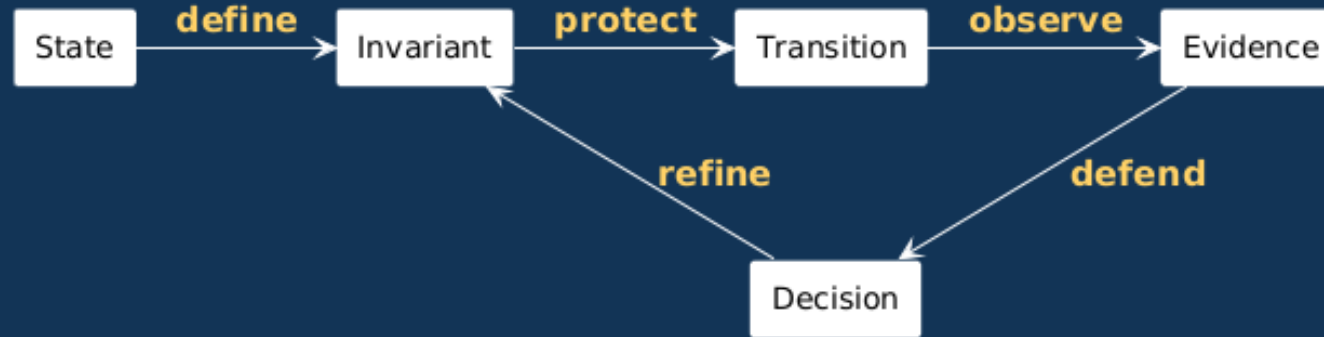
PLANTUML: PRESSURE REVEALS HIDDEN ASSUMPTIONS



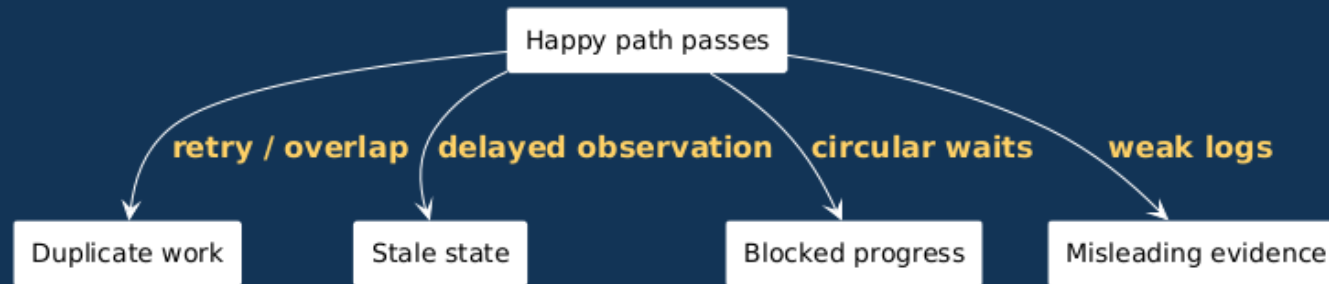
CORE IDEAS

- Cognitive load is the effort needed to understand coordination.
- Resource saturation appears when a simple design meets hardware limits.
- Scale-up adds more power to one system; scale-out adds more systems.
- A system invariant is what must stay true in every valid execution.
- A failure mode is a repeatable way the invariant is violated.
- The design response should expose the invariant and make violations measurable.

PLANTUML: INVARIANT-CENTERED DESIGN



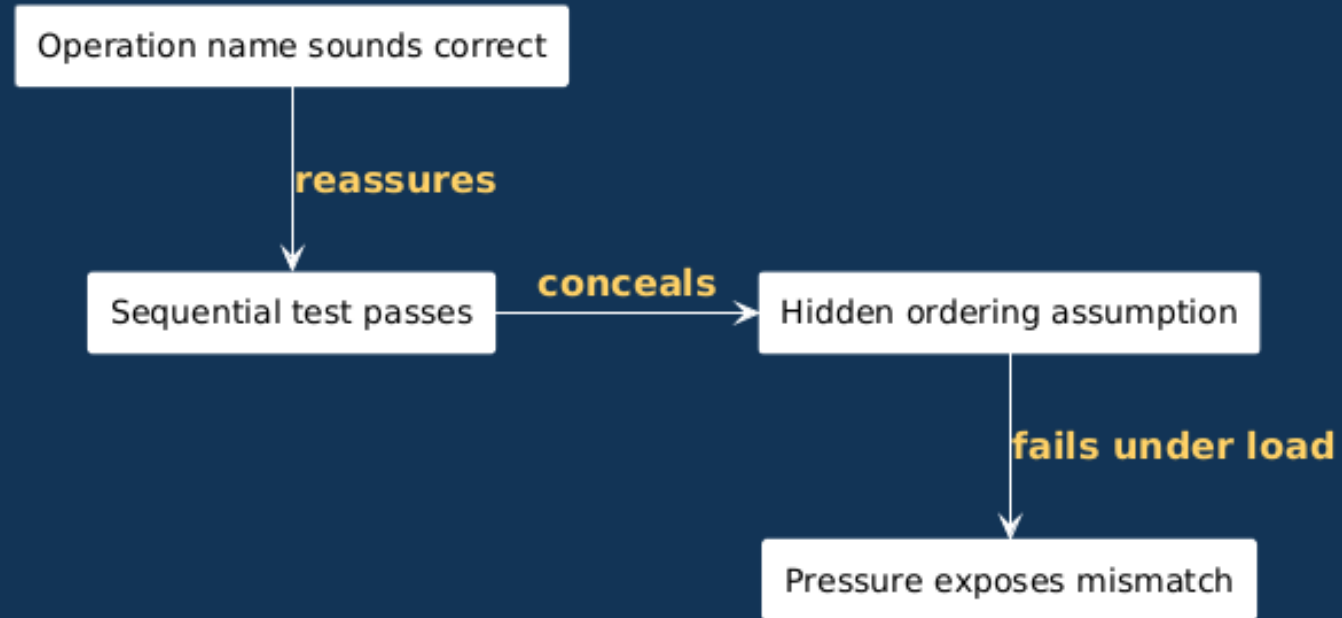
PLANTUML: COMMON FAILURE MODES



WORKED EXAMPLE

The original lecture frames this topic with examples such as a podcast domain where a simple process works in a demo but fails under overlap, delay, or concurrency pressure.

PLANTUML: NAIVE MENTAL MODEL



FAILURE SCENARIO

Under overlapping execution, delayed delivery, retry, or partial failure, the same operation can observe stale state, execute twice, arrive late, or leave the workflow stuck. The failure is not merely a bug in syntax; it is a mismatch between the design assumption and the execution model.

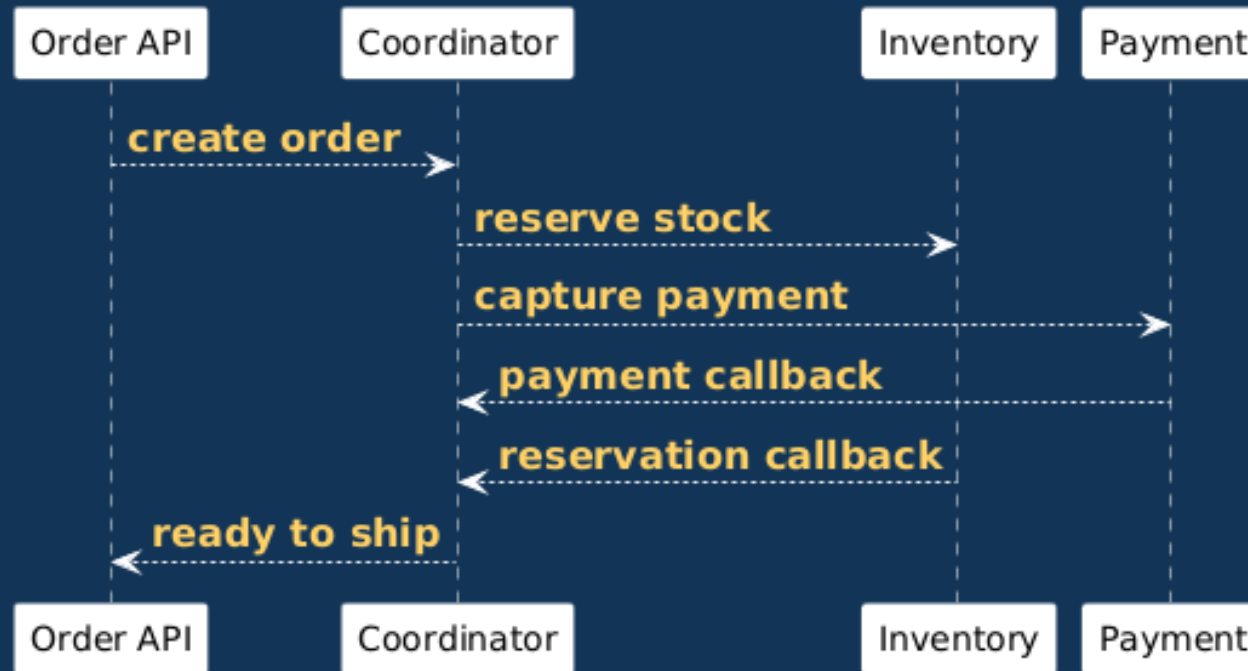
CORRECTED OR IMPROVED DESIGN DIRECTION

A safer direction is to name the invariant, make the transition observable, and add a correctness control appropriate to the failure: version checks, idempotency keys, state-machine guards, explicit coordination, retries with backoff, compensation, or reconciliation.

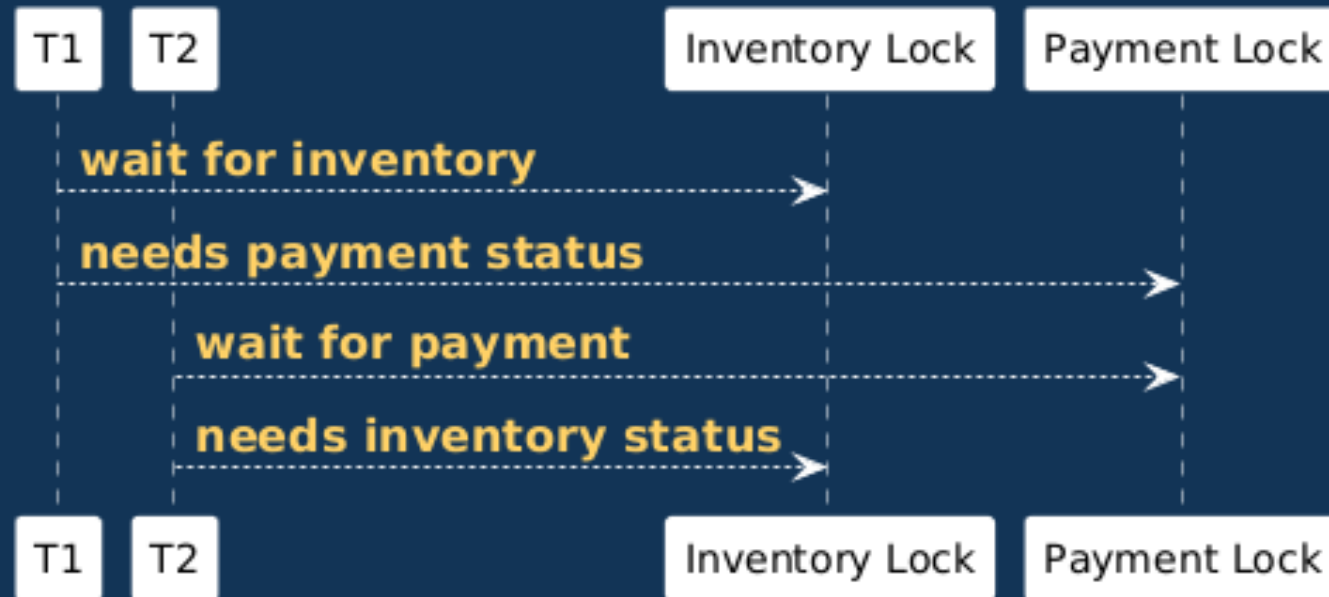
WHY THE IMPROVED DESIGN IS SAFER

The improved design is safer because it changes the problem from hoping the workflow executes in the intended order to proving that invalid interleavings are rejected, contained, or recovered with evidence.

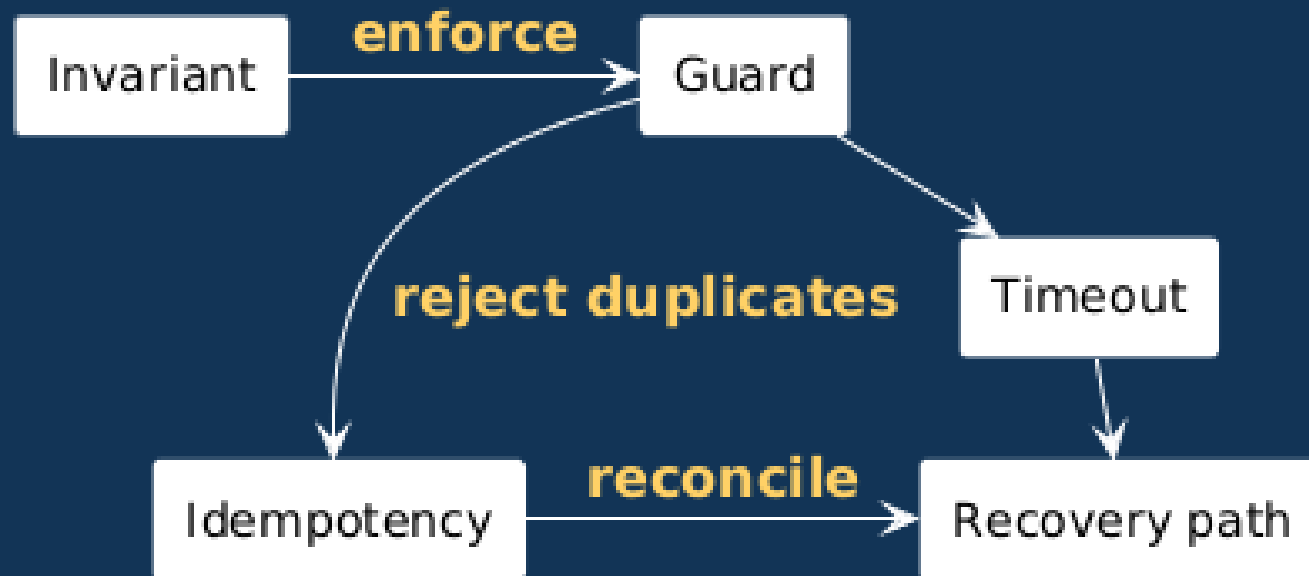
PLANTUML: WORKFLOW UNDER PRESSURE



PLANTUML: FAILURE INTERLEAVING



PLANTUML: DESIGN RESPONSE



VISUAL MODEL ANCHORS

SYSTEM INTERACTION

Actors:

- Order API
- Workflow coordinator
- Inventory service
- Payment service
- Shipping worker

Flow:

- Order API creates order-42 and requests inventory reservation

STATE TRANSITION MODEL

States:

- CREATED
- RESERVED
- PAID
- READY_TO_SHIP
- SHIPPED
- COMPENSATING

Transitions:

- CREATED → RESERVED: inventory held succeeds

FAILURE INTERLEAVING

Interleaving:

- T1: Coordinator waits for payment confirmation while holding the inventory lock
- T2: Payment callback waits for inventory status while holding the payment lock

Failure:

- Order remains stuck or ships without one prerequisite being visible in the log.

Violated invariant:

FAILURE SCENARIO

Pressure:

- Concurrent orders, delayed callbacks, and workers processing workflow steps out of order.

Observed:

- Order remains stuck or ships without one prerequisite being visible in the log.

Root cause:

- The design assumed workflow steps would be observed in the same order they were requested.

DESIGN RESPONSE

Protected property:

- Workflow progress and prerequisite ordering remain explicit under concurrent execution.

Mechanism:

- State-machine guards, ordered acquisition, timeout handling, and compensation for failed prerequisites.

Trade-off:

- More workflow metadata and recovery paths to maintain.

Diagram intent:

EVIDENCE FLOW

Claim:

- Simplicity vs. scalability is correctly handled when the order workflow preserves prerequisites and progress under pressure.

Evidence:

- Order timeline with correlation ID, lock or wait graph, state transition log, and before/after test run.

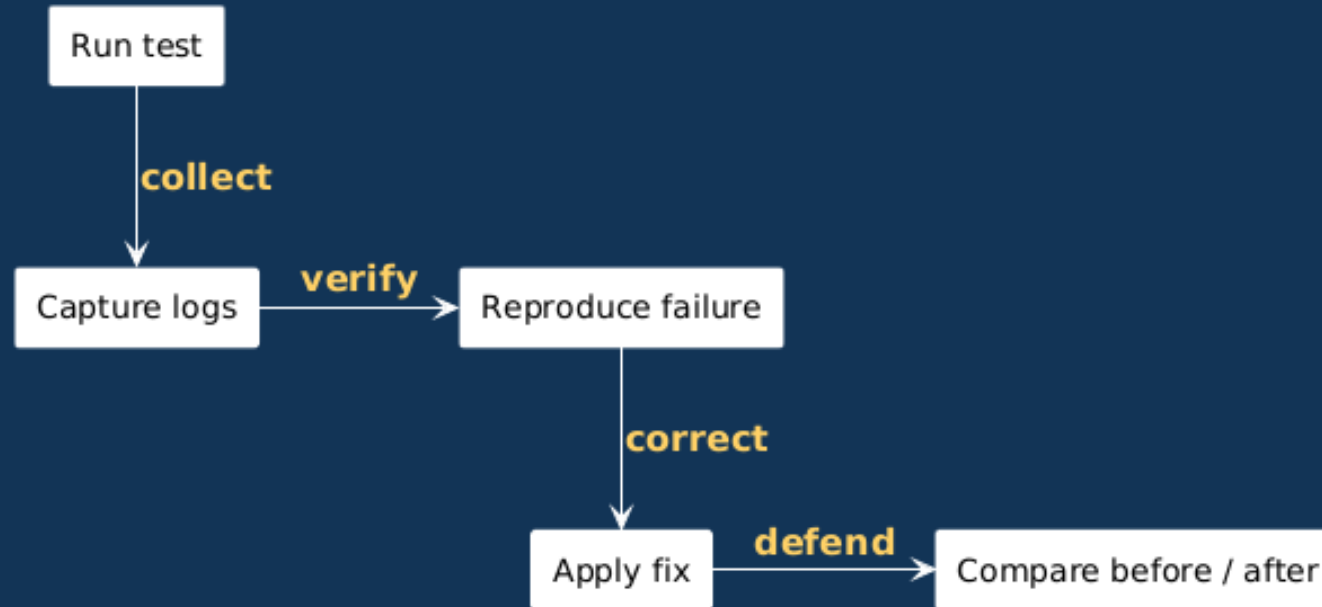
Review question:

- What exact state proves the workflow is allowed to move to the next step?

TRADE-OFFS

Approach	Strengths	Weaknesses	When to use
Simple baseline	Easy to explain and implement	Hides timing assumptions	First pass and comparison
Guarded transition	Protects the invariant directly	Needs careful state modeling	Status changes and workflow steps
Explicit coordination	Makes dependencies	Can add waiting and bottlenecks	Multi-object ordering

PLANTUML: EVIDENCE AND DECISION LOOP



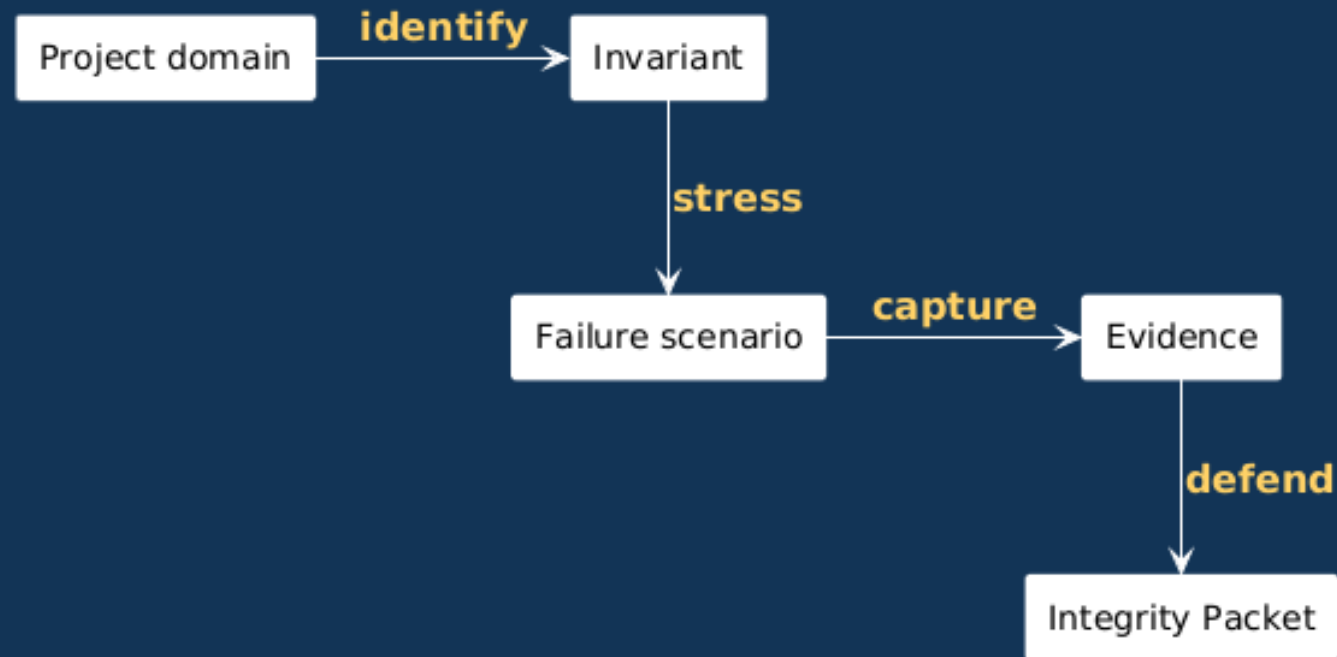
PRACTICAL APPLICATION

- Identify the invariant or progress property in your project.
- Write one sequential scenario that should succeed.
- Write one pressure scenario that could expose failure.
- Add operation IDs and entity IDs to the logs.
- Capture before/after evidence if you apply a fix.
- Update the Integrity Packet with assumption, evidence, validation, and risk.

ASSIGNMENT INTEGRATION

- This lecture supports A2 by connecting design pressure to evidence.
- Your project should show the relevant object, workflow, event, or recovery mechanism.
- Strong submissions include runnable commands, structured logs, and before/after behavior.
- The Integrity Packet should defend the design choice with concrete evidence.

PLANTUML: A2 CONNECTION



REVIEW QUESTIONS

- What invariant or progress property is most relevant here?
- What hidden assumption makes the baseline seem correct?
- How would you force the failure in a repeatable test?
- What evidence would prove the failure occurred?
- Which design mechanism would you choose first, and why?
- What trade-off does that mechanism introduce?
- What would you record in the Integrity Packet?

SUMMARY

- Simplicity vs. scalability is a way to reason about correctness under realistic execution.
- The professional task is to connect the concept to invariants, observable behavior, and trade-offs.
- A design is not mature until it can be explained, stressed, repaired, and validated with evidence.

FURTHER READING

- Search phrase: "Simplicity vs. Scalability distributed systems failure mode"
- Search phrase: "Simplicity vs. Scalability concurrency design tradeoffs"
- Topic: structured logging and correlation IDs
- Topic: invariants in stateful object-oriented systems
- Topic: coordination patterns, deadlocks, and progress guarantees